

Using Large Language Models in Critical Contexts: Robustness, Reliability, and Rule-Following by Construction and by Confinement

Alexandre HERVE

Engineering Degree, A5 — Data & AI / UX Design
ESILV — École Supérieure d'Ingénieurs Léonard de Vinci
Paris, France
alexandreherve92@gmail.com

Abstract—Large language models (LLMs) are fluent, general-purpose, and easy to embed in software. The same properties make them hazardous in critical contexts, where an incorrect output reads as confidently as a correct one and the governing failure mode is silent incorrectness rather than a visible crash. When such a system informs decisions on which human lives depend, the cost of an undetected error is not merely technical but ethical, and it raises a question of responsibility that a probabilistic component cannot answer for itself. This paper treats trustworthy deployment as a systems problem. We frame three properties a deployable system must provide and a raw model does not provide by default, namely robustness, reliability, and rule-following, and we argue that trust can be pursued along two complementary axes. The first is trust *by construction*: architectural and training-time choices, including weight tying, RMSNorm, rotary position embeddings, grouped-query attention, and domain-adapted attention, that make a model more stable, more efficient, and more amenable to inspection. The second is trust *by confinement*: constraining, grounding, verification, and human oversight applied around the model so that its residual errors become detectable and recoverable. We survey both axes, give a working formalisation of the three properties, and ground the discussion in a determinism-first decision-support case study. We report honestly what such an architecture demonstrates and what it does not, and we conclude that in any domain where errors are silent, the combination of a model built to be trustworthy and a system built to distrust it is worth more than either alone.

Index Terms—large language models, trustworthy AI, AI ethics, neuro-symbolic systems, constrained generation, hallucination, human-in-the-loop, transformer architecture, decision support

I. INTRODUCTION

Large language models are fluent, broadly knowledgeable, and trivial to wire into a software pipeline [6]. The properties that make them attractive also make them difficult to field where errors carry real-world consequences. A model that produces a plausible answer to almost any prompt will, by construction, produce a plausible answer even when the correct response is a refusal. In high-stakes settings such as medicine, aviation, finance, and defence, an incorrect recommendation does not announce itself; it is phrased with the same fluency as a correct one. The governing failure mode is therefore

not a crash but *silent incorrectness*: output that is wrong yet indistinguishable, on inspection, from output that is right.

Silent incorrectness is what turns a technical question into an ethical one. When the output of a model can influence an action that risks human life, an undetected error is not a defect to be logged but a harm that may already have occurred, and the usual reassurance that the model is “right most of the time” is precisely the wrong standard. A system that is correct on average and catastrophically, invisibly wrong on occasion is more dangerous than one that is less capable but fails loudly. This reframes the engineering target: the goal is not a model that is wrong less often, but a system in which being wrong is *detectable* and the authority to act remains with an accountable human.

We organise the requirement around three properties that a deployable system must offer and a raw LLM does not provide by default. *Robustness* is stability under perturbed or adversarial input: a small, meaning-preserving change in phrasing should not change a decision. *Reliability* is reproducibility together with calibration: the same input should yield the same output, and the system should expose when it is operating outside its competence. *Rule-following* is operation provably within hard constraints, whether those constraints are clinical protocols, aviation procedures, regulatory limits, or military doctrine.

The central organising idea of this paper is that these properties can be pursued along two complementary axes. The first is trust *by construction*: the model itself can be designed and trained to be more stable, more efficient, and more inspectable. Architectural and training-time choices, surveyed in Section IV, materially affect how a model generalises, how it behaves under input it did not see during training, and how amenable its computation is to tracing. The second is trust *by confinement*: regardless of how the model is built, the system around it can constrain, ground, verify, and oversee its output so that residual errors are caught (Section V). The two axes are not alternatives. A model built to be trustworthy still requires a system built to distrust it, because no training procedure removes the possibility of confident error.

Contributions. (i) We articulate the ethical and accountability stakes of LLMs in life-critical decision support, independent of any single application (Section II). (ii) We give a working definition of robustness, reliability, and rule-following for LLM-in-the-loop systems (Section VI). (iii) We survey both axes of trust, construction and confinement, and relate each technique to the property it targets (Sections IV and V). (iv) We ground the discussion in a determinism-first decision-support case study, reporting what such an architecture demonstrates and, with equal emphasis, what it does not (Sections VIII and IX).

II. THE STAKES: ETHICS AND RESPONSIBILITY

Before surveying techniques, it is worth stating plainly why the problem is hard in a way that exceeds engineering. Three concerns recur across every life-critical domain.

The responsibility gap. When an autonomous or semi-autonomous system contributes to a harmful outcome, it can be unclear who is answerable: the operator who acted on a recommendation, the engineer who built the system, or the organisation that fielded it. Matthias named this the responsibility gap [21]: as systems learn and act in ways their designers cannot fully predict, traditional ascriptions of responsibility come under strain. A probabilistic model cannot bear responsibility, and a fluent recommendation can lend unearned authority to a decision no person has truly examined. Closing the gap is not a matter of better accuracy; it requires that an accountable human remain genuinely in control of the decision, not merely nominally present.

Meaningful human control. The notion of *meaningful* human control distinguishes a human who can understand, question, and override a system from one who only rubber-stamps it [22]. Oversight that is procedural but not substantive provides accountability on paper and none in practice. Designing for meaningful control means surfacing what the machine produced, why, and on what evidence, so that the human’s judgement operates on information rather than on trust in the machine.

Automation bias. Human factors research shows that people tend to defer to automated recommendations even against their own better judgement, a tendency that grows as the automation appears more competent [24]. Paradoxically, a system that is usually right cultivates the complacency that makes its rare errors most dangerous, because the operator stops checking. A critical-context design must therefore treat the human’s attention as a scarce resource and direct scrutiny where it matters, rather than demanding uniform sign-off that trains the operator to stop looking.

Dual use, alignment, and regulation. Two further concerns frame the problem from outside the system. A capable decision-support tool is inherently dual-use: the same generator that compresses staff work can be repurposed or misused, so the design must consider not only whether it works but how its use is governed. Underneath this sits the value-alignment problem: a model optimised for fluency does not thereby acquire the domain’s values or legal constraints, which must

be supplied and enforced externally. Regulation is moving in the same direction: emerging frameworks classify decision-support in safety-critical and rights-affecting domains as high-risk and attach obligations of transparency, human oversight, logging, and risk management. A system designed around an inspectable backbone, marked provenance, and a genuine human gate is — not by coincidence — a system aligned with where regulation is heading; the engineering discipline and the compliance posture point the same way.

These concerns do not have purely technical solutions, but they have technical consequences. They argue for systems whose reasoning is inspectable, whose machine-produced fields are marked as such, whose errors are surfaced rather than buried, and whose human gate is designed to preserve genuine authority. The rest of the paper treats robustness, reliability, and rule-following as the engineering shadow of these ethical commitments.

III. WHY LLMs ARE HARD TO TRUST

The difficulty is structural, not incidental. A language model defines a probability distribution over next tokens, and generation samples from it conditioned on the prompt. Two consequences follow. Output is sensitive to phrasing: because the prompt shifts the distribution, semantically equivalent inputs can yield different answers, which is a robustness problem. Output can also be *fluent but false*: the model optimises for likely continuations rather than grounded truth, the mechanism behind hallucination. Ji et al. survey hallucination as content unfaithful to or unsupported by the source and document how pervasive and persistent it is across generation tasks [3]. In a critical context, this is the central hazard, because a fabricated fact is delivered with the same fluency as a correct one.

Prompting strategies mitigate but do not close these gaps. Chain-of-thought prompting improves multi-step reasoning by eliciting intermediate steps [7], and self-consistency aggregates several sampled reasoning paths to stabilise the answer [8]. Both raise average quality, yet both remain sampling over the same distribution and guarantee nothing about any individual output. For planning, the limitation is sharper still: Valmeekam et al. show on the PlanBench benchmark that strong models generate plans that look reasonable but frequently violate the problem’s own preconditions and goals when executed [5]. A model is also poorly calibrated by default: its expressed confidence is a weak signal of correctness, so the system cannot rely on the model to know when it does not know. Robustness, reliability, and rule-following therefore name three concrete gaps, none closed by a better prompt.

IV. TRUST BY CONSTRUCTION: BUILDING AND TRAINING THE MODEL

A common framing treats the LLM as a fixed black box to be wrapped. That framing is incomplete. Although the case study of Section VIII deliberately treats the model as fixed, models can in fact be designed and trained to be more trustworthy, and a complete treatment of the scientific problem must include this axis. The relevant choices are architectural

and training-time, and several have become near-standard in modern decoder-only transformers [19], [20]. Throughout this section, the three property names are used in their broad, informal senses; Section VI sharpens them. The construction techniques below should be read as shifting the model’s average behaviour in the right direction, not as delivering the precise, bounded guarantees defined there.

The first cluster of choices concerns the network itself. Each is ostensibly an efficiency or stability optimisation, but in a critical context efficiency and stability are themselves trust properties: a model that is cheaper to run leaves budget for the verification machinery around it, and a model whose training is numerically stable generalises more predictably to inputs it did not see during training.

Weight tying shares one matrix between the input embedding and the output projection, so the same vector space maps tokens in and scores them out [14]. In a model with a large vocabulary this removes a sizable block of parameters and couples the two representations so they cannot drift apart, which acts as a regulariser. The effect is a smaller, better-conditioned model, serving efficiency and, through the regularisation, reliability.

Pre-normalisation with RMSNorm normalises the input of each sub-layer rather than its output, and replaces the mean-centred layer normalisation with the cheaper root-mean-square variant that rescales without recentring [15]. Pre-norm keeps gradients well-behaved in deep stacks, so training is more stable and less sensitive to initialisation and learning rate. Stable training is the precondition for a model that behaves predictably, which is the informal root of reliability.

Rotary position embeddings (RoPE) encode position by rotating the query and key vectors by an angle proportional to their absolute position, so that the attention score between two tokens depends only on their relative distance [16]. Because position enters multiplicatively inside attention rather than as an added input, the scheme extrapolates more gracefully to sequence lengths beyond those seen in training. That graceful degradation is a form of robustness: the model breaks down less sharply when the input is longer or differently shaped than its training data.

Grouped-query attention (GQA) lets several query heads share a single key/value head, interpolating between full multi-head attention and the more aggressive multi-query variant [17]. It shrinks the key/value cache that dominates memory at inference, so a given model fits a tighter latency and footprint budget at little cost in quality. As with weight tying, the trust payoff is indirect but real: the inference headroom it frees is what lets the confinement machinery stay in the loop rather than be cut for speed.

Gated feed-forward blocks (SwiGLU) replace the plain feed-forward layer with a gated linear unit whose gate is a learned, smoothly-saturating function [18]. The gating raises quality at an equal parameter budget, one more lever that buys capability without enlarging the model and the verification cost that scales with it.

Domain-adapted attention is the least standardised of

the cluster and the one that matters most for a specialist setting. Biasing or masking the attention pattern toward the structure of the domain’s documents — their sections, fields, and cross-references — concentrates the model’s capacity on the relations that carry meaning in that domain. This improves quality on in-domain text and, because the attention is steered toward known structural relations, makes it easier to inspect where the model is looking when it produces a given field, a small contribution to the inspectability the ethics of Section II demand.

The list above concerns the network, but several training-time choices matter at least as much for trust. **Tokenisation and vocabulary** fix the granularity at which the model perceives its input; a vocabulary poorly matched to a domain’s terminology fragments key terms into many sub-tokens, which degrades both quality and the interpretability of attention over those terms. A domain-appropriate tokeniser is a quiet but real contributor to reliable behaviour on specialist text.

Data curation and instruction tuning shape what the model treats as normal. Curating the pre-training and instruction-tuning mixture toward verified, in-domain material reduces the prior mass the model places on plausible-sounding but unsupported continuations, which is the raw material of hallucination. Instruction tuning further aligns the model’s default behaviour with the response format and refusal conventions a downstream system expects, so that the model is easier to confine rather than resistant to the schema.

Alignment and calibration address how the model behaves at the margin. Preference-based alignment can teach a model to prefer cautious or abstaining answers where evidence is thin, and explicit calibration or uncertainty-quantification methods can make the model’s confidence a more honest signal of correctness. A better-calibrated model is one whose self-reported uncertainty a confinement layer can actually use to route hard cases to a human, turning the model from an opaque oracle into a component that signals when it should not be trusted.

Compression for deployment closes the loop with efficiency. Distillation and quantisation shrink a model so that it meets a latency or footprint budget, which in a critical system is what allows the verification machinery to stay in the loop rather than being cut for speed. A model that is small enough to run locally also keeps sensitive inputs inside the trust boundary, an availability and confidentiality benefit that is part of trustworthiness broadly construed.

The construction axis has a ceiling. None of these techniques removes hallucination or guarantees constraint satisfaction; they shift the distribution toward better behaviour without bounding the worst case. A doctrine-aware model is more often right, which is valuable, but “more often right” is the standard Section II rejected as sufficient. This is why the second axis is indispensable.

V. TRUST BY CONFINEMENT: BUILDING THE SYSTEM

The confinement axis takes the model as given and engineers the system so that its residual errors are caught. The

mitigations cluster into four families, each mapped to one or more target property, summarised in Table I.

Constraining narrows what the model can emit. The cheapest form is structured output validated against a schema, so malformed content is rejected before it propagates. A stronger form is grammar-constrained decoding, masking the decoder at each step to permit only tokens consistent with a target grammar [4]; practical libraries compile a schema or regular expression into a finite-state machine over the vocabulary [11], and query languages expose constraints as first-class constructs [12]. Stronger still is neuro-symbolic confinement, in which the model may operate only within a formal backbone that defines a legal artefact. Constraining chiefly buys rule-following and contributes to reliability.

Grounding ties generation to an authoritative source rather than to parametric memory. Retrieval-augmented generation conditions the model on retrieved evidence the system controls [1]; the same logic extends to fixed ontologies or single-source-of-truth references as the rule set the model must respect. Grounding attacks hallucination at its source and serves robustness and reliability.

Verification checks output after it is produced. Its mechanisms span deterministic validators and rule engines, self-correction in which the model critiques its own output [9], [10], a second model as evaluator (LLM-as-judge, which Zheng et al. show agrees strongly but imperfectly with human preference and carries biases [2]), and automated red-teaming. Verification converts an undetectable error into a detected one. A central caution, developed in Section IX, is that self-correction and LLM-as-judge place a probabilistic component in the verification path, weakening the guarantee exactly where it must be strongest.

Oversight keeps a human in authority through review gates, visible provenance marking which fields the model produced, and explicit refusal/escalation paths; deployment toolkits add programmable rails [13]. Oversight does not make the model correct; it makes the system accountable, which Section II argued is the irreducible requirement. Its design must respect the automation-bias finding: uniform sign-off trains complacency, so attention should be directed where risk concentrates.

Synthesis. Each family addresses a slice of the problem. What is comparatively rare is a system that combines all four *behind a formal backbone*, where the constraint set is an explicit, versioned formal representation of the domain rather than a prompt convention, and that ships with a measured trust signal rather than qualitative reassurance. The PlanBench result [5] is a standing reminder that, for planning, the missing piece is the external structure that catches the plans the model gets wrong.

VI. FORMALISING THE THREE PROPERTIES

We make the target properties precise enough to engineer against. Let x denote an input and let a pipeline P map it to a candidate output $y = P(x)$. Let $\mathcal{C}$ be the set of hard constraints a valid output must satisfy, and let $\text{valid}(y) \in \{0, 1\}$ indicate whether y satisfies every constraint in $\mathcal{C}$.

a) Rule-following: We require an explicit verifier $V : y \mapsto \{0, 1\}$ that is sound with respect to the constraints it encodes, $V(y) = 1 \Rightarrow \text{valid}(y)$, and that is independent of the generator. Rule-following is the property that no output reaches the human without V being applied, and that V does not depend on the same probabilistic component that produced y . A formal backbone additionally restricts the space the generator can explore, so that large classes of invalid y are unreachable by construction.

b) Reliability: Where P contains stochastic elements (sampling, or non-deterministic intermediate identifiers), reliability is reproducibility under a fixed configuration: for two executions over the same input and mode, $P(x) \equiv P'(x)$ up to a canonicalisation that ignores incidental identifiers. We operationalise $\equiv$ as byte-identical serialised artefacts after canonicalisation, which is decidable and testable.

c) Robustness: For a meaning-preserving perturbation δ , robustness bounds the change in decision so that $P(x)$ and $P(x \oplus \delta)$ agree on the load-bearing structure of the output. Because exhaustive perturbation is infeasible, robustness is approximated empirically, for example through an adversarial process that applies structured perturbations and observes whether the system degrades gracefully.

This framing has a design consequence that connects the two axes. If V must be sound and independent, then placing a probabilistic model inside V reintroduces the failure mode V exists to catch. Construction can make y better; only confinement, with a non-probabilistic V , can make a guarantee.

VII. EVALUATING TRUST: A MEASUREMENT FRAMEWORK

A recurring weakness of trustworthy-AI claims is that they are asserted rather than measured. If robustness, reliability, and rule-following are to be properties of a system rather than aspirations, each needs an operational metric. We propose a compact framework — one family of measurable signal per property — that turns the formalisation of Section VI into something a test suite can report.

Rule-following is measured at the verifier. Let $\rho = \Pr[V(y) = 1]$ be the pass rate of generated artefacts, and let the violation profile record, per constraint class, the rate and severity of failures. A severity-weighted violation score $\sum_c w_c v_c$, with weights w_c reflecting how dangerous a violation of class c is, is more informative than a single pass/fail rate, because in a critical context not all violations are equal: a cosmetic format error and a physically impossible movement should not count the same. The count of unsafe outputs caught and rejected is itself a headline number, because it is the confinement layer visibly doing the job it exists for.

Reliability is measured as reproducibility and calibration. The reproducibility rate is the fraction of repeated runs that are byte-identical after canonicalisation; under a correct caching and canonicalisation scheme this should be one, and any value below one is a defect to be hunted down rather than tolerated. Calibration is measured by an expected calibration error, comparing the model’s stated confidence with its empirical

TABLE I
THE FOUR CONFINEMENT FAMILIES, REPRESENTATIVE MECHANISMS, AND THE PROPERTY EACH CHIEFLY BUYS.

Family	Representative mechanisms	Property chiefly bought
Constraining	Schema-validated output; grammar-constrained decoding; neuro-symbolic confinement behind a formal backbone	Rule-following; reliability
Grounding	Retrieval-augmented generation; fixed ontologies; single-source-of-truth references	Robustness; reliability
Verification	Deterministic validators and rule engines; self-correction; LLM-as-judge; automated red-teaming	Rule-following
Oversight	Human-in-the-loop gates; provenance marking; refusal and escalation paths; programmable rails	Accountability

```

1:  $y \leftarrow \text{generate}(x)$  {model proposes a candidate}
2: if  $V(y) = 1$  then
3:   return  $y$  to the human gate {verified}
4: else
5:    $y' \leftarrow \text{repair}(y)$  {bounded, rule-based repair}
6:   if  $V(y') = 1$  then
7:     return  $y'$  to the human gate
8:   else
9:     record typed finding; escalate to the human
10:  end if
11: end if

```

Fig. 1. The verify-or-reject loop. No path returns an unverified artefact; the verifier V is deterministic and independent of the generator (Section VI).

accuracy across bins; a well-calibrated system is one whose confidence can be used to route uncertain cases to a human.

Robustness is measured by perturbation agreement. For a family of meaning-preserving perturbations $\{\delta_i\}$, the agreement $A = \frac{1}{n} \sum_i \mathbf{1}[P(x) \sim P(x \oplus \delta_i)]$ records how often the load-bearing structure of the output survives a perturbation, where $\sim$ denotes agreement on that structure. An adversarial variant replaces random perturbations with a process that searches for the perturbation most likely to change the decision, yielding a worst-case rather than an average-case figure.

These metrics presuppose the verifier V they are reported against. The generation procedure that makes them meaningful is the verify-or-reject loop of Figure 1: the model proposes, the deterministic verifier accepts or triggers a bounded repair, and anything that still fails is escalated to the human rather than emitted. The loop is the operational form of “the machine proposes, a person disposes”: no path returns an unverified artefact, and the human is reached either by an explicit pass or by an explicit escalation, never by default.

VIII. A CASE STUDY: DETERMINISM-FIRST DECISION SUPPORT

To make the discussion concrete, we summarise a decision-support system built as an end-of-studies project: a pipeline that converts free-form professional documents into candidate structured plans under a body of domain rules. The system is used here to illustrate the confinement axis; it deliberately treats the model as fixed and invests in the structure around it. No application-specific data is needed to follow the argument.

The work is an exploratory feasibility study, meant to establish whether this style of generation merits larger investment rather than to ship a finished product. For that reason, and because the project touches proprietary methods in a national-defence domain, specific data, algorithms, and quantitative benchmarks are subject to clearance and are described here only at the level of principle. The same constraint is part of why the construction axis is treated through the open literature rather than through this system: the system fixes its model and cannot serve as the vehicle for evaluating model-side techniques.

Architecture. The pipeline is a chain of typed stages with the model sandboxed behind a single client; no other module may call it. This is enforced structurally rather than by convention, so the model’s influence is confined to one auditable boundary. Every stage boundary is a validated schema, so a stage cannot hand the next a malformed artefact. The generative surface is deliberately small, and the components that decide acceptability contain no model at all.

Generation modes. Plan construction is offered in more than one mode. A deterministic mode decomposes the task using a formal ontology as its grammar, so it cannot emit an illegal structure. An optional enrichment mode lets the model populate bounded descriptive fields within an already-validated structure. A further, model-first mode inverts the order: the model proposes a candidate plan and the rule-based layer then validates and repairs it against the ontology. In every mode the model’s output is checked by deterministic rules before it is trusted; the modes differ only in how much of the initial draft the model contributes, never in whether it is verified.

A cheap, sound check. One verification step illustrates why structural checks catch errors that prompting alone cannot. A model may emit a spatial claim — an origin and a destination — that is linguistically fine but physically impossible within the time the plan allows. Resolving the named places to coordinates and computing the great-circle distance with the haversine formula,

$$d = 2r \arcsin \sqrt{\sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta\lambda}{2}}, \quad (1)$$

the step rejects the claim whenever the implied transit exceeds the available time, $d/v > T$, for a movement speed v . The check is sound for its class: if the inequality holds, no scheduling makes the move feasible. It catches an error the

model makes because it knows the place names as tokens but not the distances between them, an error that is physical, not linguistic, and therefore invisible to prompt-level mitigation.

Reproducibility. Intermediate identifiers are per-run unique values, so a naive cache never hits across runs. Replacing the key with a content-derived surrogate,

$$k = H(\text{hash}(\text{input}), s(\text{node}), \text{hash}(\mathcal{N}(\text{node})), \text{mode}), \quad (2)$$

where H is a cryptographic hash function (e.g. SHA-256) applied to the tuple, $s(\cdot)$ is independent of the incidental identifier, and $\mathcal{N}(\cdot)$ is the node’s neighbourhood, yields byte-identical artefacts across runs. Reproducibility is the property that makes any later measurement meaningful.

IX. OBSERVATIONS AND LESSONS

We report what the architecture demonstrates and, with equal emphasis, what it does not. Several intended experiments are scaffolded but unrun; we flag those rather than estimate them.

Reproducibility is the cleanest result. The content-derived cache key makes repeated runs byte-identical, asserted by an automated determinism test. This is unglamorous and foundational: without it there is nothing stable to measure.

A non-probabilistic verifier is a deliberate choice, not a limitation. Keeping the model out of the acceptance-deciding components follows directly from the soundness requirement of Section VI. This is the opposite of the LLM-as-judge [2] and self-correction [9], [10] reflexes; it forgoes flexibility to gain a guarantee, which is the trade a critical context should make.

Cheap physical checks earn their keep. The feasibility check of Section VIII rejects a hallucination class no prompt reliably removes, at the cost of a distance computation and a comparison.

Honest measurement is itself a result, and not all results are good. An efficiency sweep produced a high idle ratio, on the order of sixty percent of unit time, which is simply too much for a plan one would want to field. Part of that idle time may correspond to legitimate waiting inherent in the domain, but attributing all of it to benign causes would be exactly the kind of comfortable rationalisation a trustworthy-systems discipline should resist. The honest reading is that the figure is a problem, that its decomposition into benign and wasteful components is not yet established, and that the refinement of the metric and the reduction of the underlying idleness are open work rather than solved. Reporting the number as a difficulty, rather than explaining it away, is the point.

Several claims remain unmeasured. A controlled ablation isolating the contribution of each guardrail, a precision/recall characterisation of the feasibility check against human labels, and any quantified efficiency improvement from concurrency modelling are designed but not yet run, the project being an exploratory feasibility study rather than a finished system. They are stated as future measurement, not as findings.

X. DISCUSSION

Which axis carries the weight. The construction axis raises the floor: weight tying, RMSNorm, RoPE, GQA, SwiGLU, and domain-adapted attention make a model more stable, more efficient, and more often correct [14]–[18]. The confinement axis bounds the worst case: only a sound, independent verifier turns “usually right” into “never silently wrong about the constraints it encodes”. The honest conclusion is that neither suffices alone. A model built for trust still emits confident errors; a confinement layer with no help from the model spends its effort catching failures a better-built model would not have produced. The strongest systems invest in both and let the formal backbone, not the model’s fluency, carry the legal weight.

Generalisation. The pattern is domain-independent: build or adapt the model toward the domain, confine it to a sub-stage with a frozen schema, ground it in a single source of truth, verify deterministically, and keep an accountable human in authority. What changes between domains is the formal backbone — a clinical protocol, an aviation procedure, a regulatory rule set, or a doctrinal ontology — and the cheap, sound checks that falsify the domain’s characteristic hallucinations. To make this concrete, consider clinical decision support. The formal backbone becomes a clinical protocol or formulary; the single source of truth becomes a curated drug and dosage database; and the equivalent of the physical-feasibility check becomes a dosage-range or drug-interaction test, a cheap, sound rule that rejects a fluent but unsafe recommendation regardless of how confidently the model phrased it. The mechanism is identical; only the domain’s characteristic failure and the rule that falsifies it change. The same template maps onto aviation procedure-checking and onto regulatory-compliance drafting, which is the sense in which the contribution is not about military planning.

Relation to other assurance approaches. The two-axis view sits alongside several established traditions and is complementary to them rather than in competition. Formal verification of neural networks seeks mathematical guarantees about a model’s input-output behaviour [23]; it is powerful but does not yet scale to the size and open vocabulary of modern LLMs, which is precisely why confinement places the guarantee in an external, verifiable component instead of in the network. Runtime monitoring and runtime assurance wrap a component with online checks that switch to a safe fallback when a property is violated; the verify-or-reject loop of Section VII is an instance of exactly this pattern, specialised to a generative model and a symbolic backbone. Assurance-case methodology, finally, asks for a structured, auditable argument that a system is acceptably safe; the measurement framework of Section VII is intended to supply the kind of evidence such an argument needs, turning “we believe it is safe” into “here is what we measured and why it supports the claim”.

The cost of trust. Confinement is not free, and an honest account must price it. A formal backbone has to be built and maintained, which is real engineering effort that a pure-

prompting solution avoids. Verification adds latency and can over-reject, refusing a valid output because the encoded rules are stricter or coarser than reality, which is a false-positive cost that erodes operator trust in the opposite direction from automation bias. The human gate, the strongest guarantee, is also the scarcest resource and the most expensive bottleneck. The trade is worth making when the cost of a silent error is high and irreversible, which is the defining feature of a critical context; it is a poor trade when errors are cheap, visible, and easily undone, where the overhead buys little.

When confinement is the wrong tool. It follows that the approach has a domain of applicability rather than universal merit. Where the task is open-ended or creative and admits no crisp notion of a “valid” output, there is no sound verifier to build, and confinement degenerates into arbitrary rejection. Where the domain’s rules are themselves uncertain or contested, encoding them as hard constraints can be worse than leaving judgement to a human, because it lends the rules a false air of authority. The technique is at its best exactly where this paper places it: a domain with explicit, stable, checkable constraints and a high cost of silent error. Naming that boundary is part of taking the approach seriously.

Threats to validity. Reproducibility is operationalised as byte-identity after canonicalisation, a proxy for the broader notion of reliability that omits calibration. The efficiency observation comes from a small sweep rather than a controlled experiment, so it supports a qualitative claim and not a precise figure. The system has not been evaluated by a domain professional, so the human-control claims remain architectural rather than empirically validated. The verifier is sound only for the constraints it encodes, and post-hoc constraining is weaker than token-level decoding [4], [11].

Limitations. The determinism-confines-the-model stance has a ceiling: it makes the model safe by making it small, so the system inherits the coverage gaps of its deterministic backbone, and where the domain is genuinely ambiguous no validation substitutes for the human gate. The construction axis has the complementary ceiling already noted: it improves the average without bounding the worst case.

XI. CONCLUSION

Trust in a critical-context LLM system comes from two complementary sources: a model built and trained to be stable, efficient, and inspectable, and a system built to constrain, ground, verify, and oversee what the model produces. The first axis raises how often the model is right; the second axis ensures that when it is wrong, the error is detectable and an accountable human retains control. The ethical stakes — the responsibility gap, meaningful human control, and automation bias — are not solved by accuracy; they are addressed by architectures whose reasoning is inspectable and whose authority rests with a person. A determinism-first case study shows that such an architecture is buildable today, and that its trust is in principle measurable, while being candid that important measurements remain open and that at least one efficiency result is a problem rather than a success. The

argument does not depend on those numbers. Where errors are silent, trust has to be engineered into the structure around the model and into the model itself, in tandem: the deployable unit is a system that can detect its own mistakes and keep a person in command, not a model judged only by how seldom it errs.

REFERENCES

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, arXiv:2005.11401.
- [2] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *NeurIPS*, 2023, arXiv:2306.05685.
- [3] Z. Ji et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023, doi:10.1145/3571730.
- [4] S. Geng et al., “Grammar-constrained decoding for structured NLP tasks without finetuning,” in *EMNLP*, 2023, arXiv:2305.13971.
- [5] K. Valmeekam et al., “PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change,” in *NeurIPS Datasets and Benchmarks Track*, 2023, arXiv:2206.10498.
- [6] T. B. Brown et al., “Language models are few-shot learners,” in *NeurIPS*, 2020, arXiv:2005.14165.
- [7] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *NeurIPS*, 2022, arXiv:2201.11903.
- [8] X. Wang et al., “Self-consistency improves chain of thought reasoning in language models,” in *ICLR*, 2023, arXiv:2203.11171.
- [9] A. Madaan et al., “Self-Refine: Iterative refinement with self-feedback,” in *NeurIPS*, 2023, arXiv:2303.17651.
- [10] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” in *NeurIPS*, 2023, arXiv:2303.11366.
- [11] B. T. Willard and R. Louf, “Efficient guided generation for large language models,” 2023, arXiv:2307.09702.
- [12] L. Beurer-Kellner et al., “Prompting is programming: A query language for large language models,” in *Proc. ACM Program. Lang. (PLDI)*, 2023, doi:10.1145/3591300.
- [13] T. Rebecq et al., “NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” in *EMNLP (System Demonstrations)*, 2023, arXiv:2310.10501.
- [14] O. Press and L. Wolf, “Using the output embedding to improve language models,” in *EACL*, 2017, arXiv:1608.05859.
- [15] B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *NeurIPS*, 2019, arXiv:1910.07467.
- [16] J. Su et al., “RoFormer: Enhanced transformer with rotary position embedding,” 2021, arXiv:2104.09864.
- [17] J. Ainslie et al., “GQA: Training generalized multi-query transformer models from multi-head checkpoints,” in *EMNLP*, 2023, arXiv:2305.13245.
- [18] N. Shazeer, “GLU variants improve transformer,” 2020, arXiv:2002.05202.
- [19] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” 2023, arXiv:2302.13971.
- [20] S. Raschka, *Build a Large Language Model (From Scratch)*. Shelter Island, NY: Manning, 2024.
- [21] A. Matthias, “The responsibility gap: Ascribing responsibility for the actions of learning automata,” *Ethics and Information Technology*, vol. 6, no. 3, pp. 175–183, 2004, doi:10.1007/s10676-004-3422-1.
- [22] F. Santoni de Sio and J. van den Hoven, “Meaningful human control over autonomous systems: A philosophical account,” *Frontiers in Robotics and AI*, vol. 5, art. 15, 2018, doi:10.3389/frobt.2018.00015.
- [23] G. Katz et al., “Reluplex: An efficient SMT solver for verifying deep neural networks,” in *Computer Aided Verification (CAV)*, 2017, arXiv:1702.01135.
- [24] R. Parasuraman and D. H. Manzey, “Complacency and bias in human use of automation: An attentional integration,” *Human Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi:10.1177/0018720810376055.